World Sim Game - Code Review Notes for Codex

给新的 code review tab / Codex 使用的架构 review 规则、500 行限制、模块边界和可复制 prompt。

> **核心意图：**请把这份文档当作 code review 的工作说明。目标不是加新玩法，而是让代码更可维护、更容易继续扩展，并且保证每个代码文件不超过 500 行。

1. Code Review 最高优先级规则

规则	要求
不加新功能	review 阶段只看结构、bug、可维护性，不新增玩法。
不改游戏行为	除非明确是 bug，否则不要改变生成逻辑、扩张逻辑、战争逻辑。
每次只改一类问题	不要一次同时重构 UI、世界生成、战争、渲染。
保持可编译	每一步改完都必须能 build。
先给计划再动手	先列出要改哪些文件、为什么改、怎么验证。
代码文件不超过 500 行	超过就必须按职责拆分。
不要创建万能头文件	不要把所有东西继续塞进一个 `.h`。
不要让模块互相乱 include	`world` 不依赖 `ui`，`simulation` 不依赖 `render`。
渲染只负责画	`render` 不应该修改模拟状态。
UI 只发命令	UI 不应该直接改地图内部算法。
世界生成不认识文明	`world generation` 不应该依赖 civilization simulation。

2. 500 行文件规则

```
No source or header file should exceed 500 lines.

If a file grows beyond 500 lines, split it by responsibility, not by arbitrary line count.
```

文件类型	建议上限
`.c` 文件	500 行以内
`.h` 文件	300 行以内
数据表文件	尽量 500 行以内，太大就拆表
generated / raw asset manifest	可以例外，但必须说明

超过 500 行时如何拆

情况	拆法
`simulation.c` 太长	拆成 `city.c`、`province.c`、`summary.c`、`civilization.c`
`world_gen.c` 太长	拆成 `climate.c`、`terrain_query.c`、`world_gen.c`
`render.c` 太长	拆成 `map_render.c`、`panel_render.c`、`icon_render.c`、`colors.c`
`ui.c` 太长	拆成 `input.c`、`panel.c`、`sliders.c`
`game_types.h` 太长	拆成 `world_types.h`、`sim_types.h`、`ui_types.h`、`render_types.h`
`game_tables.h` 太长	拆成 `geography_rules.h`、`climate_rules.h`、`ecology_rules.h`、`resource_rules.h`

> **关键原则：**拆文件不是为了变短，而是为了让每个文件只有一个职责。

3. 推荐模块边界

模块	应该负责	不应该负责
`core`	基础类型、常量、工具函数、全局状态或 `GameState`	世界生成、UI、渲染、战争
`world`	地理、气候、生态、资源、地形查询	文明扩张、外交、战争、UI
`sim`	文明、城市、行省、扩张、外交、战争	绘图、按钮、slider
`render`	地图绘制、边界绘制、图标绘制、panel 绘制	修改地图、修改文明、推进回合
`ui`	输入、按钮、slider、表单、用户命令	直接生成地形、直接改变战争结果
`game`	初始化、主循环、调度	具体算法细节

推荐依赖方向

```
core
  -> world
  -> sim
  -> render / ui
  -> game
```

更直白一点：

```
world 不 include sim
world 不 include ui
world 不 include render

sim 可以读 world
render 可以读 world 和 sim
ui 可以发命令
game 负责调度所有模块
```

4. Review 时重点看什么

4.1 文件是否职责清楚

每个文件都应该能用一句话解释。

文件	合格描述
`world_gen.c`	只负责生成世界
`climate.c`	只负责气候分类
`province.c`	只负责行省形状、边界距离、城市位置修复
`city.c`	只负责城市创建、升级、城市属性
`summary.c`	只负责国家和行省统计缓存
`war.c`	只负责军团、战争判定、战争结算
`diplomacy.c`	只负责关系状态、贸易契合、紧张度
`render.c`	只负责绘制调度
`icon_render.c`	只负责图标加载和绘制
`ui.c`	只负责窗口消息分发和 UI 命令

> 如果一个文件负责城市、行省、国家、缓存、扩张、人口、战争、UI 一点点，那它已经开始变成代码章鱼，应该拆。

4.2 是否有全局变量到处被改

问题	更好的方向
UI 直接改 `ocean_slider`	UI 改 `WorldGenConfig`
render 直接依赖一堆全局变量	render 接收 `const GameState *`
simulation 到处读写 `world`、`civs`、`cities`	逐步改成 `GameState *game`
world_gen 直接读 UI slider	world_gen 读取 `WorldGenConfig`

暂时不用一次全改，但 review 时要标记出来。

4.3 是否有万能 include

```
#include "world/world.h"
```

如果 `world.h` 同时 include 了 `world_gen.h` 和 `simulation.h`，那它不是 world header，而是大礼包 header。

情况	建议
一个 header include 太多模块	拆小
world header include simulation	不允许
sim header include render	不允许
render header include ui	通常不应该
所有文件都 include `game_types.h`	说明 `game_types.h` 太胖

4.4 是否有行为和命名不一致

例子	问题
`trade_fit` 里扣掉 aggression	贸易契合不再是纯贸易契合
`province_id == city_id`	短期能用，长期会限制迁都、拆省、割地
`RESOURCE_COUNT` 同时用于 resource feature	名字可能混淆真正资源和特殊资源点
`military` 是 metric，`aggression` 也是旧 trait	需要明确一个是能力，一个是倾向
`world_gen.c` 里有颜色函数	世界生成和渲染混在一起

5. Reviewer 检查顺序

顺序	检查内容
1	是否能编译
2	文件是否超过 500 行
3	是否有模块职责混乱
4	header 是否过大
5	include 依赖是否反向
6	render 是否修改模拟状态
7	UI 是否直接改世界生成内部
8	world_gen 是否依赖 simulation
9	simulation 是否还可以继续拆
10	外交战争是否超出当前需求
11	数据表是否可维护
12	README build command 是否同步

6. 给 code review tab 的直接指令

可以直接复制下面这段给新的 code review tab：

```
Please review this C Windows GDI game codebase.

Main goal:
Find architecture, module-boundary, maintainability, and file-size problems.
```

```
Do not add new gameplay.
Do not rewrite algorithms.
Do not make code changes unless I explicitly ask.
First produce a review report.

Important project rule:
No .c or .h file should exceed 500 lines.
If a file exceeds 500 lines, suggest a responsibility-based split.

Review priorities:

1. Check whether each file has a single clear responsibility.
2. Check whether any file exceeds 500 lines.
3. Check whether game_types.h is too large or acts as a god header.
4. Check whether global state is overused.
5. Check whether world generation depends on simulation, rendering, or UI.
6. Check whether simulation depends on rendering or UI.
7. Check whether rendering mutates game state.
8. Check whether UI directly mutates world generation internals instead of using config or
commands.
9. Check whether headers include too much.
10. Check whether module names match what the code actually does.
11. Check whether province, city, diplomacy, war, and expansion logic should be split
further.
12. Check whether the build command includes all source files.
13. Check whether README and source layout are consistent.

For each issue, report:

- File
- Problem
- Why it matters
- Suggested solution
- Whether it is urgent
- Whether it changes gameplay or is refactor-only

Do not suggest new features unless they are necessary to fix architecture.
Do not invent new gameplay systems.
Keep recommendations small and step-by-step.
```

7. 给 Codex 改代码时的执行指令

当准备让 Codex 真的改代码时，使用这段：

```
Refactor only. No gameplay changes.

Apply the following rule:
No source or header file should exceed 500 lines.

Before editing:
1. List all files over 500 lines.
2. For each one, propose a responsibility-based split.
3. Do not edit yet until the split plan is clear.

When editing:
1. Move code only.
2. Do not rewrite algorithms.
```

```
3. Do not rename public fields unless needed for compilation.
4. Keep behavior unchanged.
5. Keep the game compiling.
6. Update the build command or Makefile.
7. Do one module at a time.

Preferred split order:
1. Split game_types.h into smaller type headers.
2. Split simulation.c into city, province, summary, civilization, and simulation
coordinator.
3. Split world_gen.c into world generation, terrain query, climate, and render colors.
4. Split render.c into map_render, panel_render, icon_render, and colors.
5. Split ui.c into input, panel, sliders, and UI command handling.

Do not add diplomacy, war, new resources, new terrain, or new UI features in this
refactor.
```

8. 文件超过 500 行时的拆分模板

```
File: src/simulation.c
Current issue:
This file mixes city creation, province shaping, province cache, country summaries,
civilization creation, and monthly simulation.

Suggested split:
- sim/city.c + sim/city.h
- sim/province.c + sim/province.h
- sim/summary.c + sim/summary.h
- sim/civilization.c + sim/civilization.h
- simulation.c keeps only high-level monthly/yearly update coordination

Behavior change:
None. Move code only.

Acceptance:
- Game compiles
- Existing city/province behavior remains unchanged
- No source file exceeds 500 lines
```

9. 最终一句话

```
Please review like an architecture reviewer, not a feature designer.
The goal is cleaner module boundaries, files under 500 lines, and safer future
development.
```

> **重点：**现在要它当建筑质检员，不是让它当世界造物主。否则它可能又会开始加联盟系统、附庸系统、外交 UI 和别的新玩法。